INT161 · Basic Backend Development

# Pagination and DTO

ส่งรายการขนาดใหญ่ทีละหน้าด้วย page, limit และ offset และกำหนดรูปแบบ response ด้วย DTO

Why Pagination | page, limit, offset | LIMIT, OFFSET and COUNT | Response Structure | Data Transfer Object | Mapper and apiResponse | อภิธานศัพท์

01

## Why Pagination

ลองนึกถึงร้านค้าออนไลน์ที่มีสินค้าร้อยล้านรายการ ถ้า **back end** โหลดทั้งหมดแล้วส่งให้ **front end** จะไม่รอด รายการจึงมาทีละหน้า และ front end ขอหน้าที่ต้องการ

- **response** ใหญ่ทำให้หน่วยความจำของ **server** เต็ม database ทำงานหนัก และส่งผ่าน network ไม่ทัน
- หน้าทำให้ **client** ได้ขนาดที่คาดเดาได้ และ front end เดาขนาดหน้าที่เหมาะสมได้
- front end ทำ **pagination** ได้ก็ต่อเมื่อ back end รองรับ
- มีสองแบบ: offset-based ที่เราใช้ และ **cursor-based** แบบ **cursor** เร็วกว่าสำหรับหน้าลึกมาก แต่ไล่ไปตามลำดับเท่านั้น กระโดดไปหน้า 500 ไม่ได้ ส่วน **offset** กระโดดได้ เราจึงใช้แบบนี้

02

## page, limit, offset

client ส่ง **query** parameter สองตัว เช่น GET /api/products?page=2&**limit**=20

- page: หน้าที่ client ต้องการ limit: จำนวนรายการต่อหน้า
- back end คำนวณ offset: จำนวนรายการที่ต้องข้ามเพื่อเริ่มหน้าที่ถูกต้อง offset = (page - 1) x limit

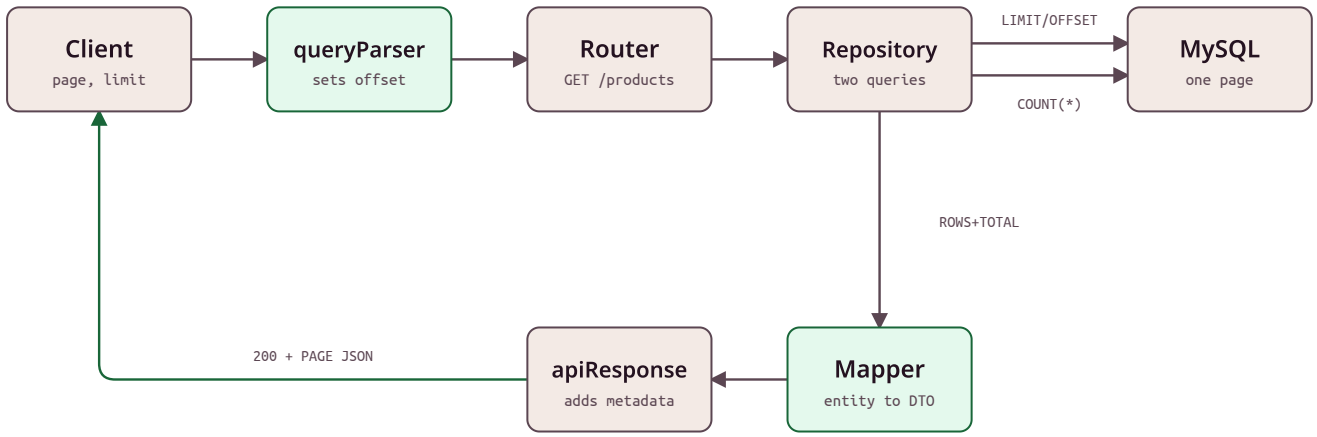
page = 2 (ที่ client ส่งมา)

limit 10 **20** 50 500

มี 138 รายการรวม server ใช้ page = 2 และ limit = 20

| offset | SQL | items |
| --- | --- | --- |
| `(2 - 1) x 20 = 20` | `LIMIT 20 OFFSET 20` | `21 to 40` |

| totalPages | hasNextPage | hasPrevPage |
| --- | --- | --- |
| `ceil(138 / 20) = 7` | `true` | `true` |




- ตรวจ input page ต้องไม่ต่ำกว่า 1 ถ้าได้ 0 หรือติดลบให้เป็น 1 ถ้าไม่ส่ง limit ใช้ค่าเริ่มต้น 20 จำกัด limit ไม่เกิน 100 เพื่อไม่ให้ใครขอหน้ายักษ์
- อ่าน page และ limit ใน **middleware** ตัวเดียว ไม่ต้องอ่านในทุก **router**

pagination-middleware.js

```
// middlewares/pagination-middleware.js
export const queryParser = () => (req, res, next) => {
  const query = req.query || {};
  if (query.page == undefined || query.limit == undefined) {
    req.pagination = null; // the client did not ask for pages
    return next();
  }
  const page = Math.max(parseInt(query.page), 1); // never below 1
  const limit = Math.min(parseInt(query.limit), 100); // never above 100
  const offset = (page - 1) * limit;
  req.pagination = { page, limit, offset };
  next();
};
```

app.js

```
import { queryParser } from './middlewares/pagination-middleware.js';
const app = express();
app.use(express.json());
app.use(queryParser()); // before the routers
```

> ⚠ **ลำดับ**
>
> app.use(queryParser()) ต้องมาก่อน router เพราะ router อ่าน req.pagination ถ้า middleware ทำงานทีหลังจะยังไม่มีค่า

03

## LIMIT, OFFSET and COUNT

สอง query อันหนึ่งดึงหน้านั้น อีกอันนับทั้งหมด เพื่อให้ response บอกได้ว่ามีกี่หน้า

repositories/product-repository.js

```
const COUNT_ALL = `SELECT COUNT(*) as total FROM ${TABLE_NAME}`;
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`;
async function count() {
  const [[{ total }]] = await db.query(COUNT_ALL);
  return total;
}
export async function findAll(pagination) {
  if (!pagination) {
    const [rows] = await db.query(FIND_ALL_SQL);
    return rows;
  }
  const SQL = FIND_ALL_SQL + ' LIMIT ? OFFSET ?';
  const { limit, offset } = pagination;
  const [rows] = await db.query(SQL, [limit, offset]);
  rows.totalItems = await count(); // the layers above need it for the metadata
  return rows;
}
```

- LIMIT และ OFFSET ก็เป็น **placeholder** ค่าอยู่ใน array: limit ก่อน offset ทีหลัง
- ถ้าไม่มี object pagination แปลว่า client ไม่ได้ขอ ให้ส่งทั้งหมด ใช้ได้กับรายการสั้น เช่น รายชื่อจังหวัด แต่ตารางใหญ่ห้ามทำแบบนี้
- ใช้ **prepared statement** เสมอ ช่วยป้องกัน **SQL injection** ด้วย
- ถ้าเรียงหรือกรองตามคอลัมน์ไหน ให้ทำ **database index** ให้คอลัมน์นั้น ถ้าไม่มี ตารางใหญ่จะช้า
- ข้อเสียของ offset: เพื่อเริ่มที่รายการ 10,000 database ยังต้องอ่านผ่าน 10,000 รายการแรก หน้าลึกมากจึงช้า และเป็นเหตุผลที่มี **cursor pagination**

04

## Response Structure

front end ต้องการมากกว่าข้อมูล ต้องรู้ว่าอยู่ตรงไหน หน้าไหน มีทั้งหมดกี่รายการ มีหน้าถัดไปหรือก่อนหน้าไหม ซึ่งใช้ควบคุมปุ่มเปลี่ยนหน้า

Response with pages

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "total": 138,
    "page": 2,
    "limit": 20,
    "totalPages": 7,
    "hasNextPage": true,
    "hasPrevPage": true
  },
  "timestamp": "2026-10-08T15:00:00.000Z"
}
```

- hasPrevPage เป็น false แปลว่าไม่มีหน้าก่อนหน้า front end จึงปิดปุ่ม Previous ส่วน hasNextPage ใช้กับ Next แบบเดียวกัน
- ใช้รูปแบบเดียวกันทุก **endpoint**: success, data และ pagination (ถ้ามี) สไลด์หนึ่งแผ่นใช้ชื่อ totalItems และ hasPreviousPage แต่ code ใช้ total และ hasPrevPage ข้อสอบตัวอย่างใช้ชื่อตาม code

05

## Data Transfer Object

table คือ **entity** และมีทุกคอลัมน์ บางคอลัมน์มีไว้ให้ database เช่น last update บางอันที่ front end ไม่ต้องใช้ บางอันห้ามออกเด็ดขาด เช่น password hash **DTO** คือสำเนาที่ปรับรูปแล้วที่ส่งแทน

ส่ง entity | ส่ง DTO

```
{
  "id": ...,
  "name": ...,
  "email": ...,
  "role": ...
}
```

password_hash และ created_at ไม่ออกจาก server รูปแบบเหมือนกันทุก endpoint แม้เป็น hash ก็ไม่ควรส่ง เพราะ hash ถูกโจมตีแบบ brute force ได้

- ทำให้ **API** สม่ำเสมอ และแยก API ออกจากโมเดลของ database
- เปลี่ยนชื่อ field ได้ JavaScript และ **JSON** มักใช้ **camelCase** แต่คอลัมน์ใน database อาจเป็น **snake_case** back end เป็นคนแปลง
- รวมหลาย entity ได้ เช่น customer พร้อม order ใน response เดียว front end เรียกครั้งเดียวแทนที่จะสองครั้ง
- ใช้ได้ทั้งสองทาง สำหรับ **request** ให้รับเฉพาะ field ที่จำเป็น ไม่ใช่ทุกคอลัมน์ของ entity

**Summary**

ไม่กี่ field สำหรับรายการ รายการสินค้าต้องการชื่อและราคา ไม่ใช่คำอธิบายยาว

**Detail**

ครบทุกอย่างสำหรับรายการเดียว แสดงเมื่อผู้ใช้คลิกเข้าไปดูสินค้า

> **จะ map ที่ไหน**
> ทำที่ **service layer** หรือ **controller** ก็ได้ JavaScript ตัดหรือเพิ่ม field ได้ง่าย แต่ก็ควรเขียนไว้ใน class ที่ใช้ซ้ำได้ ไม่ใช่เขียนแทรกทุก endpoint

06

## Mapper and apiResponse

หนึ่ง **mapper** ต่อหนึ่ง entity product มี product mapper customer มี customer mapper ไม่ใช่ class กลางตัวเดียว

mappers/product-mapper.js

```
// mappers/product-mapper.js
export default class ProductMapper {
  static toSummaryDTO(entity) {
    return {
      id: entity.productCode,
      name: entity.productName,
      scale: entity.productScale,
      retailPrice: entity.MSRP,
    };
  }
  static toDetailDTO(entity) {
    const { MSRP, buyPrice, ...data } = entity; // pull out the two we rename
    return { ...data, retailPrice: MSRP, price: buyPrice };
  }
  static map(entity, viewType) {
    return viewType === 'full' ? this.toDetailDTO(entity) : this.toSummaryDTO(entity);
  }
  static maps(entities, viewType) {
    const { totalItems } = entities;
    const dtos = entities.map((e) => this.map(e, viewType));
    if (totalItems) dtos.totalItems = totalItems;
    return dtos;
  }
}
```

- client เลือก view ด้วย query parameter ?view=full ค่าเริ่มต้นของรายการคือ **summary** ค่าเริ่มต้นของรายการเดียวคือ full
- maps วนทุก entity และเก็บ totalItems ที่ **repository** แนบมาไว้

ขั้นสุดท้ายคือฟังก์ชันเดียวที่สร้าง JSON ทั้งก้อน ทุก endpoint จึงหน้าตาเหมือนกัน

mappers/api-response.js

```
// mappers/api-response.js
import httpStatus from 'http-status';
export async function apiResponse(res, data, pagination = null, status = httpStatus.OK) {
  if (!pagination) {
    return res.status(status).json({ success: true, data, timestamp: new Date().toISOString() });
  }
  const { limit, page } = pagination;
  const total = data.totalItems;
  return res.status(status).json({
    success: true,
    data,
    pagination: {
      total, page, limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
      hasPrevPage: page > 1,
    },
    timestamp: new Date().toISOString(),
  });
}
```

routes/product-route.js

```
// routes/product-route.js
router.get("/", async (req, res) => {
  const pagination = req.pagination;
  const viewType = req.query.view || "summary";
  const objects = await service.getAllObjects(pagination, viewType);
  apiResponse(res, objects, pagination);
});
router.get("/:id", async (req, res) => {
  const viewType = req.query.view || "full";
  const object = await service.getObject(req.params.id, viewType);
  apiResponse(res, object);
});
```

- router ไม่สร้าง JSON เอง แต่ส่งผลลัพธ์และ pagination ให้ apiResponse ซึ่งเขียน response ทั้งแบบมีและไม่มี **metadata** ของหน้า
- เพราะทุกอย่างผ่านฟังก์ชันเดียว JSON จึงสม่ำเสมอแน่นอน
- ลองใน **browser** หรือ **Postman**: page 3 กับ limit 3 จะได้ hasNextPage และ hasPrevPage เป็น true ขอ limit ใหญ่ขึ้น จำนวนหน้าจะลดลง

07

## อภิธานศัพท์

**API**
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

**back end**
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

**browser**
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

**camelCase**
: คำต่อกันโดยใช้ตัวใหญ่ เช่น userName เป็นธรรมเนียมของตัวแปร JavaScript

**client**
: ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

**controller**
: layer ที่รับ HTTP request และส่ง response แล้วส่งต่องานจริงให้ layer อื่น

**cursor**
: แทนที่จะข้าม ให้จำว่าหน้าก่อนหน้าจบตรงไหนแล้วทำต่อจากตรงนั้น เร็วสำหรับหน้าลึก แต่กระโดดไปหน้า 500 ไม่ได้

**cursor pagination**
: แทนที่จะข้าม ให้จำว่าหน้าก่อนหน้าจบตรงไหนแล้วทำต่อจากตรงนั้น เร็วสำหรับหน้าลึก แต่กระโดดไปหน้า 500 ไม่ได้

**cursor-based**
: แทนที่จะข้าม ให้จำว่าหน้าก่อนหน้าจบตรงไหนแล้วทำต่อจากตรงนั้น เร็วสำหรับหน้าลึก แต่กระโดดไปหน้า 500 ไม่ได้

**database index**
: โครงสร้างที่ database เก็บไว้เพื่อหาแถวเร็ว ควรทำ index ให้คอลัมน์ที่ใช้เรียงหรือกรอง

**DTO**
: สำเนาของ entity ที่ปรับรูปสำหรับ API มีเฉพาะ field ที่ client ต้องการ ในรูปแบบที่ตกลงกัน

**endpoint**
: method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

**entity**
: ข้อมูลใน database ตามที่เก็บ: แถวของ table พร้อมทุกคอลัมน์

**front end**
: ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

**JSON**
: JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

**limit**
: จำนวนรายการต่อหนึ่งหน้า client ส่งมาเป็น limit และ server กำหนดเพดาน

**mapper**
: class ที่แปลง entity เป็น DTO เพื่อให้การแปลงอยู่ที่เดียวและใช้ซ้ำได้

**metadata**
: ข้อมูลเกี่ยวกับข้อมูล ใน response ของหน้า: เลขหน้า limit จำนวนรวม และจำนวนหน้ารวม

**middleware**
: ฟังก์ชัน (req, res, next) ที่ทำงานระหว่าง request กับ route handler แก้ request ตอบเอง หรือส่งต่อได้

**offset**
: จำนวนรายการที่ต้องข้ามก่อนเริ่มหน้า: (page - 1) x limit

**pagination**
: ส่งรายการขนาดใหญ่ทีละหน้า เพื่อให้ response เล็กและเร็ว

**placeholder**
: เครื่องหมาย ? ใน prepared statement ค่าจะถูกใส่แทน ? ตามลำดับ

**Postman**
: เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

**prepared statement**
: SQL ที่เขียน ? ไว้ตรงที่จะใส่ค่า แล้วส่งค่าแยกต่างหาก database จะไม่เข้าใจผิดว่าค่าเป็นคำสั่ง

**query**
: ส่วนของ URL หลัง ? เช่น page=2&size=10 ใช้กรองหรือแบ่งหน้ารายการ

**repository**
: layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

**request**
: สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

**response**
: สิ่งที่ server ส่งกลับ: status code, header และ body

**router**

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

**server**

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

**service layer**

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

**snake_case**

คำต่อกันด้วย underscore เช่น user_name ใช้บ่อยกับคอลัมน์ใน database ไม่ใช้กับ URI

**SQL injection**

การโจมตีที่ข้อมูลจากผู้ใช้ถูกต่อเข้า SQL แล้วเปลี่ยนความหมายของคำสั่ง placeholder ป้องกันได้

**summary**

DTO สองขนาด: summary มีไม่กี่ field สำหรับรายการ detail มีครบสำหรับรายการเดียว